

Nome do Aluno:	RA:
-----------------------	------------

Curso: Ciência da Computação	
Disciplina: Paradigma de Linguagens de Programação	Campus: Passo Fundo
Professor (a): Luciano Rodrigo Ferretto	Data: ____ / ____ / 2026

CARTÃO DE RESPOSTAS

Assinatura do Aluno	ATENÇÃO! Instruções para preenchimento 1. Não dobre ou rasure este cartão de respostas 2. Marque apenas uma resposta por questão 3. Todas as questões devem ser respondidas com caneta azul ou preta 4. Nas dissertativas, caso o espaço não seja suficiente, pode-se usar o verso da folha. 5. Preencher totalmente o espaço conforme o modelo abaixo: ☒ Forma correta de preenchimento
----------------------------	--

Questões Objetivas

01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
B	B	B	B	B	B	B	B	B	B	B	B	B	B	B
C	C	C	C	C	C	C	C	C	C	C	C	C	C	C
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D
E	E	E	E	E	E	E	E	E	E	E	E	E	E	E

Questões Dissertativas nas próximas folhas

Orientações:

- A prova teórica é individual e sem consulta ao material.
- **Lembre-se de colocar as respostas das questões objetivas no cartão da primeira página, e nas páginas seguintes as respostas das questões dissertativas.**
 - **NÃO SERÁ CORRIGIDO NADA ALÉM DESSAS 2 (DUAS) PRIMEIRAS PÁGINAS.**
- Questões Objetivas: 6,2
- Questões Dissertativas: 0,8

UMA ÓTIMA PROVA A TODOS!



A large, faint, light gray letter 'A' watermark is centered on a background of horizontal ruling lines. The watermark is a simple, bold, sans-serif capital letter. The background consists of approximately 20 horizontal lines, evenly spaced, in a light gray color. The watermark 'A' is positioned in the center of the page, spanning across several of the ruling lines.

1) Durante a implementação de um sistema, dois desenvolvedores apresentam soluções diferentes para o mesmo problema. O primeiro descreve uma sequência de comandos que altera variáveis e controla explicitamente o fluxo da execução. O segundo utiliza funções para transformar os dados, procurando evitar alterações no estado compartilhado.

Considerando as características dos paradigmas de programação, analise as afirmações.

- I.** A primeira solução apresenta características do paradigma imperativo.
- II.** A segunda solução apresenta características associadas ao paradigma funcional.
- III.** O paradigma funcional procura reduzir a dependência de alterações de estado e efeitos colaterais.
- IV.** O paradigma imperativo não permite a utilização de funções durante a construção de programas.

Assinale a alternativa correta.

- A)** Apenas I e II estão corretas.
- B)** Apenas I, II e III estão corretas.
- C)** Apenas I, III e IV estão corretas.
- D)** Apenas II, III e IV estão corretas.
- E)** I, II, III e IV estão corretas.

2) Um sistema possui uma classe Funcionario com atributos protegidos contra acesso direto. As alterações de seu estado são realizadas por métodos que controlam os valores recebidos. Além disso, uma classe Gerente especializa Funcionario e redefine determinado comportamento herdado.

Considerando os conceitos da Programação Orientada a Objetos, as características apresentadas estão relacionadas, respectivamente, a:

- A)** Abstração e encapsulamento.
- B)** Encapsulamento e polimorfismo.
- C)** Herança e abstração.
- D)** Polimorfismo e herança.
- E)** Abstração e herança.

3) Uma aplicação precisa determinar se determinadas relações entre pessoas são verdadeiras. Para isso, o programador cadastra fatos sobre indivíduos e estabelece regras que representam relações entre eles. Depois, realiza consultas para obter informações que não foram cadastradas diretamente.

Nesse cenário, o mecanismo utilizado para encontrar respostas a partir dos fatos e regras está relacionado ao:

- A)** controle de fluxo do paradigma procedural.
- B)** mecanismo de inferência do paradigma lógico.
- C)** processo de composição de funções do paradigma funcional.
- D)** mecanismo de herança do paradigma orientado a objetos.
- E)** processo de encapsulamento do paradigma orientado a objetos.

4) Uma equipe utiliza uma linguagem que permite combinar diferentes formas de construção de programas. Em determinadas partes do sistema são utilizados objetos e classes, enquanto em outras são utilizadas funções e operações para transformação de coleções.

Considerando o conceito de linguagens multiparadigma, analise as afirmações.

- I.** Uma linguagem multiparadigma pode oferecer recursos associados a diferentes paradigmas.
- II.** A utilização de mais de um paradigma pode ocorrer dentro de um mesmo projeto.
- III.** A adoção de diferentes paradigmas pode permitir diferentes formas de abordar problemas.
- IV.** Uma linguagem multiparadigma exige que cada projeto utilize somente um paradigma por vez.

Assinale a alternativa correta.

- A)** Apenas I e II estão corretas.
- B)** Apenas I, II e III estão corretas.
- C)** Apenas I, III e IV estão corretas.
- D)** Apenas II, III e IV estão corretas.
- E)** I, II, III e IV estão corretas.

5) Considere uma função que recebe uma lista de valores e retorna uma nova lista contendo apenas os valores que atendem a determinada condição. A função não altera a lista original e produz o mesmo resultado sempre que recebe a mesma entrada.

Considerando as características da programação funcional, essa situação está principalmente relacionada aos conceitos de:

- A) herança, polimorfismo e encapsulamento.
- B) fatos, regras e mecanismo de inferência.
- C) funções puras, imutabilidade e transformação de dados.
- D) procedimentos, variáveis globais e alteração de estado.
- E) classes abstratas, interfaces e especialização de objetos.

6) Durante a revisão de um sistema, um desenvolvedor encontra uma classe denominada SistemaUtil contendo métodos para enviar e-mails, calcular impostos, validar documentos e gerar relatórios. Embora o código funcione corretamente, diferentes alterações no sistema frequentemente exigem modificações nessa classe.

Considerando os princípios de design estudados, analise as afirmações.

- I. A classe apresenta baixa coesão por concentrar responsabilidades distintas.
- II. A situação pode dificultar a manutenção do sistema.
- III. A separação das responsabilidades em componentes mais específicos pode melhorar a organização do código.
- IV. A concentração de várias responsabilidades caracteriza necessariamente baixo acoplamento.

Assinale a alternativa correta.

- A) Apenas I e II estão corretas.
- B) Apenas I, II e III estão corretas.
- C) Apenas I, III e IV estão corretas.
- D) Apenas II, III e IV estão corretas.
- E) I, II, III e IV estão corretas.

7) Uma equipe está desenvolvendo um sistema e decide implementar diversas funcionalidades que não fazem parte dos requisitos atuais, pois acredita que elas poderão ser necessárias futuramente. Com o passar do tempo, parte dessas funcionalidades nunca é utilizada.

Considerando os princípios KISS, YAGNI e DRY, essa decisão está mais diretamente relacionada à violação do:

- A) princípio KISS, por aumentar a quantidade de classes utilizadas.
- B) princípio DRY, por criar funcionalidades em diferentes módulos.
- C) princípio YAGNI, por implementar necessidades que ainda não existem.
- D) princípio KISS, por utilizar abstrações em vez de implementações concretas.
- E) princípio DRY, por manter diferentes responsabilidades na mesma classe.

8) Uma classe Relatorio possui métodos responsáveis por consultar dados, calcular estatísticas, formatar informações e enviar o relatório por e-mail. Durante a evolução do sistema, mudanças na forma de envio de e-mails exigem alterações na mesma classe que realiza os cálculos.

Considerando os princípios SOLID, a situação está principalmente relacionada ao:

- A) SRP, pois a classe possui diferentes responsabilidades.
- B) OCP, pois a classe não utiliza nenhuma interface.
- C) LSP, pois a classe possui métodos com diferentes comportamentos.
- D) ISP, pois a classe deveria implementar várias interfaces menores.
- E) DIP, pois toda classe com mais de um método viola inversão de dependência.

9) Considere o seguinte cenário:

Uma classe Pagamento depende diretamente de PagamentoCartao. Para permitir que futuramente sejam utilizados pagamentos por cartão, PIX ou boleto, o desenvolvedor cria uma interface FormaPagamento e faz Pagamento receber essa abstração.

A alteração realizada tem como principal objetivo:

- A) aumentar a quantidade de dependências concretas existentes no sistema.
- B) permitir que a classe de alto nível dependa de uma abstração.

- ..
- C) garantir que todas as formas de pagamento compartilhem a mesma implementação.
D) impedir que novas formas de pagamento sejam adicionadas ao sistema.
E) concentrar todas as regras de pagamento em uma única implementação concreta.
-

10) Analise as situações a seguir relacionadas aos princípios SOLID.

- I. Uma classe possui diferentes motivos independentes para sofrer alterações.
II. Uma implementação derivada não pode substituir adequadamente a classe-base esperada pelo cliente.
III. Uma interface exige que seus implementadores forneçam métodos que não utilizam.
IV. Uma nova funcionalidade exige modificar repetidamente uma classe existente para incluí-la.

As situações apresentadas estão associadas, respectivamente, a:

- A) SRP, LSP, ISP e OCP.
B) OCP, LSP, SRP e DIP.
C) SRP, DIP, ISP e LSP.
D) DIP, LSP, OCP e SRP.
E) ISP, SRP, DIP e OCP.
-

11) Uma aplicação monolítica possui módulos de clientes, produtos e pedidos. Em determinada situação, o módulo de produtos passa a receber uma quantidade de requisições muito superior aos demais módulos. Na arquitetura atual, a equipe precisa aumentar os recursos da aplicação inteira.

Considerando as características de microservices, analise as afirmações.

- I. A arquitetura monolítica agrupa diferentes funcionalidades em uma unidade de implantação.
II. Microservices podem permitir que um serviço seja escalado independentemente de outros.
III. A adoção de microservices pode aumentar a complexidade relacionada à comunicação entre serviços.
IV. Microservices eliminam completamente a necessidade de infraestrutura para implantação.

Assinale a alternativa correta.

- A) Apenas I e II estão corretas.
B) Apenas I, II e III estão corretas.
C) Apenas I, III e IV estão corretas.
D) Apenas II, III e IV estão corretas.
E) I, II, III e IV estão corretas.
-

12) Uma equipe está dividindo uma aplicação em três microservices: Cliente, Pedido e Produto. Cada serviço deverá ser responsável pelos dados relacionados ao seu próprio domínio. O serviço de pedidos, entretanto, precisa consultar informações pertencentes ao serviço de produtos.

Considerando o **Database per Service Pattern**, qual abordagem está mais alinhada ao princípio estudado?

- A) O serviço de pedidos deve consultar diretamente as tabelas do banco do serviço de produtos.
B) Todos os serviços devem utilizar as mesmas tabelas para evitar duplicação de informações.
C) Cada serviço deve controlar seus dados, utilizando mecanismos de comunicação para acessar informações de outros serviços.
D) O serviço de produtos deve conceder acesso administrativo ao banco para os demais serviços.
E) Todos os serviços devem compartilhar o mesmo schema para garantir consistência imediata.
-

13) Uma equipe utiliza Flyway para controlar a evolução do banco de dados. O projeto possui as migrações V1__criar_usuario.sql, V2__adicionar_email.sql e V3__criar_indice.sql.

Durante a execução da aplicação, a equipe precisa determinar quais alterações já foram aplicadas ao banco.

Nesse contexto, a tabela de histórico do mecanismo de migração tem como principal finalidade:

- A) armazenar cópias dos dados antes da execução de cada script.
B) registrar as migrações executadas e controlar seu histórico de aplicação.
C) substituir o controle de versão utilizado pelo código-fonte da aplicação.
D) armazenar somente as consultas SQL que apresentaram maior tempo de execução.
E) controlar as permissões dos usuários que acessam as tabelas do banco.
-

14) Uma organização possui vários microservices executados nos ambientes de desenvolvimento, testes e produção. Para evitar que os valores de conexão com banco, URLs e níveis de log sejam mantidos diretamente no código, a equipe utiliza configurações externas e diferentes perfis de execução.

Considerando **Externalized Configuration** e **Configuration Profiles**, analise as afirmações.

- I. A configuração pode ser fornecida externamente ao código da aplicação.
- II. Perfis podem organizar diferentes conjuntos de configurações.
- III. O mesmo código pode ser executado com configurações diferentes conforme o ambiente.
- IV. O uso de perfis exige que cada ambiente possua uma implementação diferente da aplicação.

Assinale a alternativa correta.

- A) Apenas I e II estão corretas.
- B) Apenas I, II e III estão corretas.
- C) Apenas I, III e IV estão corretas.
- D) Apenas II, III e IV estão corretas.
- E) I, II, III e IV estão corretas.

15) Uma equipe possui configurações espalhadas em vários projetos de microservices. Quando uma URL de integração precisa ser alterada, os desenvolvedores precisam localizar e modificar diversos arquivos diferentes. Para reduzir essa dispersão, a organização decide utilizar um serviço central responsável por disponibilizar as configurações aos microservices.

Considerando os patterns estudados, essa solução está diretamente relacionada ao:

- A) Database Migration Pattern, pois centraliza alterações do banco de dados.
- B) Database per Service Pattern, pois centraliza os dados de todos os serviços.
- C) Centralized Configuration Pattern, pois centraliza o gerenciamento das configurações.
- D) Service Discovery Pattern, pois centraliza o registro das instâncias dos serviços.
- E) Externalized Configuration Pattern exclusivamente, pois toda configuração centralizada deve estar fora da aplicação.

16) Considere o código Java a seguir:

```
public class CalculadoraArea {
    public double calcularArea(Object forma) {
        if (forma instanceof Circulo) {
            Circulo c = (Circulo) forma;
            return Math.PI * c.raio * c.raio;
        } else if (forma instanceof Retangulo) {
            Retangulo r = (Retangulo) forma;
            return r.largura * r.altura;
        }
        return 0;
    }
}
```

O sistema inicialmente trabalha com círculos e retângulos. Posteriormente, surge a necessidade de calcular a área de uma nova forma geométrica, como um triângulo. Para atender ao novo requisito, seria necessário modificar o método calcularArea() e adicionar uma nova condição para tratar a nova forma.

Com base nos princípios SOLID estudados na disciplina:

- a) Identifique o princípio SOLID diretamente relacionado ao problema apresentado e explique por que a implementação atual viola esse princípio.
- b) Proponha uma solução de projeto que permita adicionar novas formas geométricas sem precisar modificar a classe CalculadoraArea a cada nova forma. Apresente as principais classes/interfaces envolvidas e explique como a solução proposta resolve o problema.